Evaluation and Metrics

Davide Falessi

Data Mining

Practical Machine Learning Tools and Techniques

Slides for Chapter 5, Evaluation

of *Data Mining* by I. H. Witten, E. Frank,
M. A. Hall and C. J. Pal

Credibility: Evaluating what's been learned

- Issues: training, testing, tuning
- Predicting performance: confidence limits
- Holdout, cross-validation, bootstrap
- Hyperparameter selection
- Comparing machine learning schemes
- Predicting probabilities
- Cost-sensitive evaluation
- Evaluating numeric prediction
- The minimum description length principle
- Model selection using a validation set

Evaluation: the key to success

- How predictive is the model we have learned?
- Error on the training data is *not* a good indicator of performance on future data
- Simple solution that can be used if a large amount of (labeled) data is available:
  - Split data into training and test set
- However: (labeled) data is usually limited
  - More sophisticated techniques need to be used

Training and testing I

- Natural performance measure for classification problems: *error rate*
  - *Success*: instance's class is predicted correctly
  - *Error*: instance's class is predicted incorrectly
  - Error rate: proportion of errors made over the whole set of instances

Training and testing II

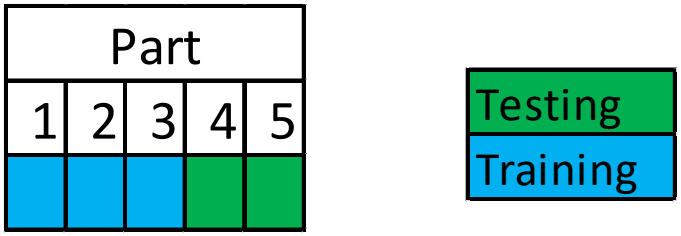
- *Test set*: independent instances that have played no part in formation of classifier
  - Assumption: both training data and test data are representative samples of the underlying problem
- Test and training data may differ in nature
  - Example: classifiers built using customer data from two different towns *A* and *B*
  - To estimate performance of classifier from town *A* in completely new town, test it on data from *B*

Making the most of the data

- Once evaluation is complete, *all the data* can be used to build the final classifier
- Generally, the larger the training data the better the classifier (but returns diminish)
- The larger the test data the more accurate the error estimate
- *Holdout* procedure: method of splitting original data into training and test set
  - Dilemma: ideally both training set *and* test set should be large!

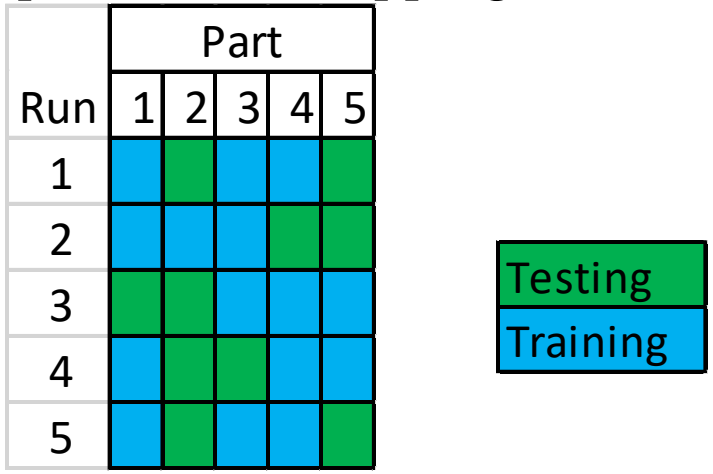
Holdout estimation

- What should we do if we only have a single dataset?
- The *holdout* method reserves a certain amount for testing and uses the remainder for training, after shuffling
  - Usually: one third for testing, the rest for training
- Problem: the samples might not be representative
  - Example: class might be missing in the test data
- Advanced version uses *stratification*
  - Ensures that each class is represented with approximately equal proportions in both subsets



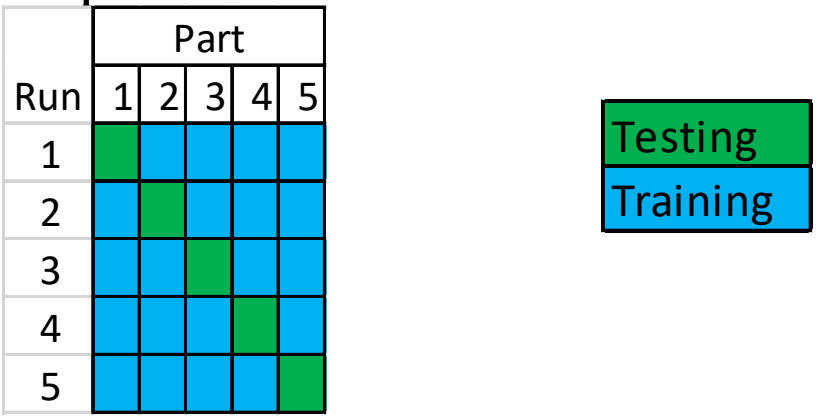
Repeated holdout method

- Holdout estimate can be made more reliable by repeating the process with different subsamples
  - In each iteration, a certain proportion is randomly selected for training (possibly with stratificiation)
  - The error rates on the different iterations are averaged to yield an overall error rate
- This is called the *repeated holdout* method
- Still not optimum: the different test sets overlap
  - Can we prevent overlapping?



Cross-validation

- *K-fold cross-validation* avoids overlapping test sets
  - First step: split data into k subsets of equal size
  - Second step: use each subset in turn for testing, the remainder for training
  - This means the learning algorithm is applied to k different training sets
- Often the subsets are stratified before the cross-validation is performed to yield stratified k-fold cross-validation
- The error estimates are averaged to yield an overall error estimate; also, standard deviation is often computed



More on cross-validation

- Standard method for evaluation: stratified ten-fold cross-validation
- Why ten?
  - Extensive experiments have shown that this is the best choice to get an accurate estimate
  - There is also some theoretical evidence for this
- Stratification reduces the estimate's variance
- Even better: repeated stratified cross-validation
  - E.g., ten-fold cross-validation is repeated ten times and results are averaged (reduces the variance)

Leave-one-out cross-validation

- Leave-one-out:
  a particular form of *k*-fold cross-validation:
  - Set number of folds to number of training instances
  - I.e., for *n* training instances, build classifier *n* times
- Makes best use of the data
- Involves no random subsampling
- Very computationally expensive (exception: using lazy classifiers such as the nearest-neighbor classifier)
- Might overestimate the training set.

Leave-one-out CV and stratification

- Disadvantage of Leave-one-out CV: stratification is not possible
  - It *guarantees* a non-stratified sample because there is only one instance in the test set!

Run	Instance									
	1	2	3	4	5	6	7	8	9	10
1										
2										
3										
4										
5										
6										
7										
8										
9										
10										



The bootstrap

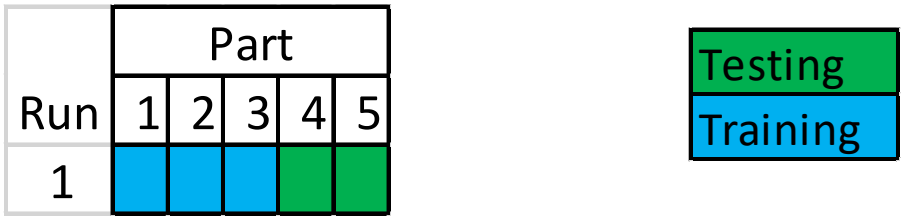
- *CV uses sampling without replacement*
  - The same instance, once selected, can not be selected again for a particular training/test set
- The *bootstrap* uses sampling *with replacement* to form the training set
  - Sample a dataset of *n* instances *n* times *with replacement* to form a new dataset of *n* instances
  - Use this data as the training set
  - Testing set: the instances from the original dataset that do not occur in the new training set.

Time-series Validation Techniques

- One characteristic of a technique is its ability to preserve the temporal order of data; i.e., preventing the testing set to have data antecedent to the training set.
- These techniques are generally called time-series and must be used when the data is time-sensitive, i.e., when data shows a pattern related to time.
- Non-time-series techniques aim to estimate the classifier accuracy on a context similar to the data available, e.g., future releases or similar project.
  - Suppose you have a dataset related to a project of five releases and that the proportion of defective classes in releases four and five is significantly lower than in the previous three releases. In a realistic context, to predict release four, the classifier is trained on releases one to three; a technique that measures the accuracy of the classifier on release four would bias results if data of release five is used during training. If this is true, then, to provide realistic results, it is important to ensure that train set data precede the test set data.

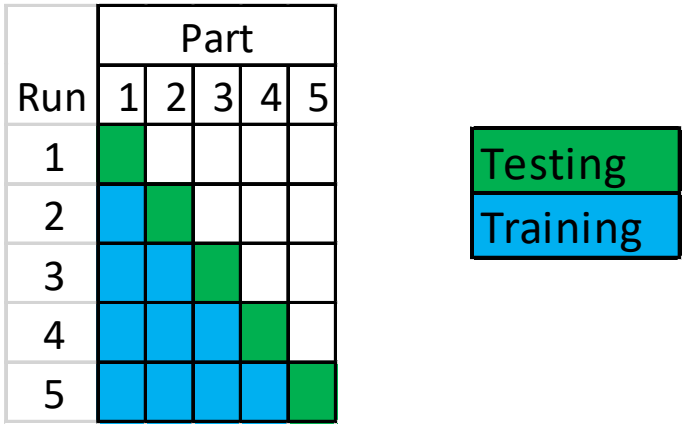
Ordered holdout

- The split preserves the order of data.



WALK-FORWARD

- Walk-forward is a widely used time-series technique.
  - In walk-forward, the dataset is divided into parts, i.e., the smallest units that can be ordered, e.g., a release of a project.
  - Then, parts are chronologically ordered, and in each run, all data available before the part to predict is used as the training set, and the part to predict is used as test-set.
  - Afterward, the model accuracy is computed as the average among runs. The number of runs is equal to one less than the number of parts.



Summary on validation techniques



There are multiple techniques to use the data to evaluate the accuracy of a model.



All techniques have pros and cons.



In science we try to understand if a technique is particularly better than others, in a particular context.



You need to understand if the instances are independent or not to decide to use a non-timeseries or not.



It is always a good idea to use more than one validation technique!

Performance metrics

- A binary classifier produces output with two class values or labels, such as Yes/No, 1/0, Positive/Negative for given input data.
- A dataset used for performance evaluation is called a test dataset.

Performance metrics

- The *confusion matrix*:
  - TP: predicted positive and actually positive
  - FP: predicted positive and actually negative
  - TN: predicted negative and actually negative
  - FN: predicted negative and actually positive

		Predicted class	
		Yes	No
Actual class	Yes	True positive	False negative
	No	False positive	True negative

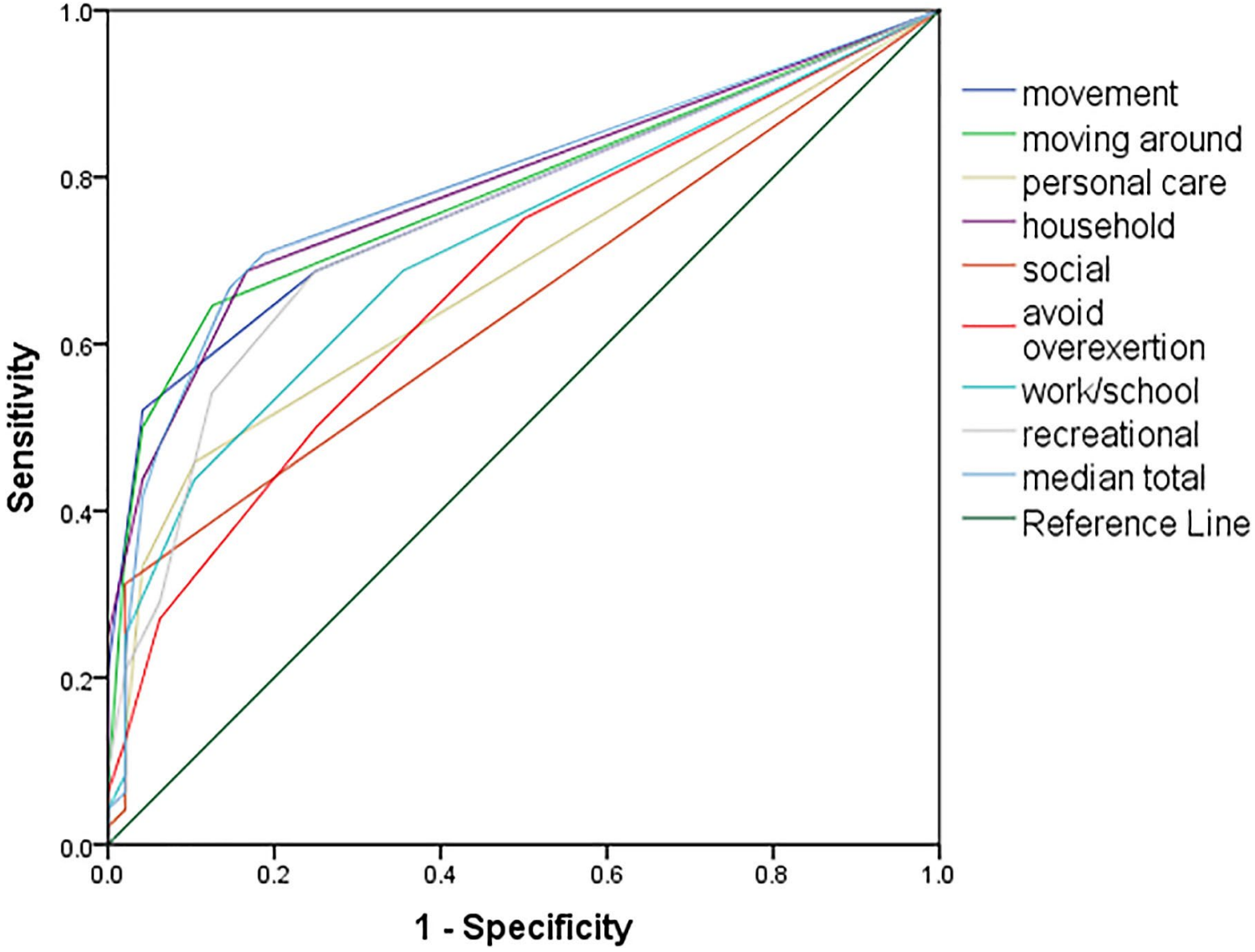
Precision and Recall

- **Recall**: how many of the positives you have been able to classify
  - TP/(TP + FN)
- **Precision**: how many times you have correctly classified an instance as positive
  - TP/(TP+FP)
- **Accuracy**: (Tx)/(Tx + Fx)
- These metrics are domain dependent.
  - High Recall might be very easy for a dummy classifier when data is 99% positive.
- **Kappa**: how many times you have been more accurate than a dummy classifier

ROC CURVES

- The main disadvantage of all previous metrics is that they are threshold dependent.
- **ROC curve**: plotting the true positive rate (TPR) (Y axis) against the false positive rate (FPR) (X axis) at various threshold settings.
- Area under the ROC curve (**AUC**): probability that randomly chosen positive instance is ranked above randomly chosen negative one.

A SAMPLE ROC CURVE



Summary on accuracy metrics



There are multiple metrics to measure the accuracy of a model.



All accuracies have pros and cons.



In science we try to understand if a metric is particularly more important than others in a particular context.



No single metric can provide you a sufficient understanding of the accuracy of a model.



It is always a good idea to interpret more than one accuracy metric.